Internal LMS – AI Build Spec (React + Node + Supabase)

Oct 26, 2026 ·

This is the single source of truth for building an internal Learning Management System (LMS) with React, Node.js and Supabase. Hand it to an AI coding agent one phase at a time (Section 8) and it has everything needed to build, with no guessing.

1. How the AI must use this doc

Rules for the AI builder (non-negotiable):

1. Build only what this doc specifies. If something is ambiguous, pick the simplest option, write the assumption in `docs/ASSUMPTIONS.md`, and continue.
2. Build one phase at a time (Section 8). Do not start the next phase until the current phase's acceptance checklist passes.
3. Use TypeScript everywhere (frontend and backend). Strict mode on.
4. Validate every request body with Zod. Never trust client input.
5. All authorization is enforced on the server (Express middleware plus Supabase RLS). Hiding a button in React is never security.
6. No public sign-up anywhere: no sign-up page, no sign-up endpoint. Users are created only by an Admin.
7. Never hardcode secrets. Read all config from environment variables; commit `.env.example` only.
8. Every list endpoint is paginated. Every table has `created_at`; mutable tables have `updated_at`.
9. Write at least one test per endpoint (happy path plus one permission-denied case) using Vitest and Supertest.
10. Every screen needs loading, empty and error states.

Definition of done for any feature: migration applied, API implemented and validated, RLS policy written, UI built (responsive, accessible), tests passing, acceptance criteria in the feature spec ticked.

Glossary:

Term	Meaning
Admin	Internal staff who create users and courses, and handle complaints
Teacher	Creates course content, grades work, posts announcements
Student	Consumes content, submits work, sees own marks
Course	A subject, taught by one or more teachers, with enrolled students
Enrollment	The link between a student and a course

2. Tech stack and architecture

The React SPA talks to a Node/Express API; the API talks to Supabase (Postgres, Auth, Storage, Realtime). The browser uses Supabase directly only for Realtime chat subscriptions; everything else goes through the API.

Layer	Choice	Notes
Frontend	React 18 + Vite + TypeScript	React Router v6, TanStack Query, React Hook Form + Zod, Tailwind CSS, shadcn/ui
Backend	Node 20 + Express + TypeScript	Zod validation, Helmet, CORS allow-list, express-rate-limit, pino logging
Database	Supabase Postgres	SQL migrations in `supabase/migrations`, RLS enabled on every table
Auth	Supabase Auth (email + password)	Sign-ups disabled in dashboard; admin creates users with the service-role key
File storage	Supabase Storage	Private buckets: `resources`, `submissions`, `complaint-attachments`; access via signed URLs (60 s expiry)
Realtime	Supabase Realtime	Used for classroom chat and live announcement badges
Email	Supabase Auth SMTP	Password reset and new-user invite emails

Layer	Choice	Notes
Testing	Vitest, Supertest, Playwright	Unit, API, and one E2E happy path per feature
Hosting (suggested)	Vercel (web), Render/Railway (API)	Any host works; keep it 12-factor

Auth flow:

1. The user logs in on React via `POST /api/auth/login` (email, password). The API calls Supabase `signInWithPassword` and returns `access_token`, `refresh_token`, and the profile (with `role`).
2. React stores the session in memory plus a refresh token in an httpOnly cookie (preferred) and attaches `Authorization: Bearer <access_token>` to every call.
3. Express middleware `requireAuth` verifies the JWT with Supabase, loads `profiles.role`, and attaches `req.user = { id, role }`. `requireRole('teacher')` guards role-specific routes.
4. A user with `profiles.is_active = false` is rejected even with a valid token.

Environment variables:

```
# apps/api/.env.example
PORT=4000
CLIENT_ORIGIN=http://localhost:5173
SUPABASE_URL=
SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=   # server only, never exposed to the browser
PASSWORD_RESET_REDIRECT=http://localhost:5173/reset-password

# apps/web/.env.example
VITE_API_URL=http://localhost:4000/api
VITE_SUPABASE_URL=
VITE_SUPABASE_ANON_KEY=
```

3. Roles and permissions

Three roles are stored in `profiles.role`: `admin`, `teacher`, `student`. A teacher only touches courses they are assigned to; a student only sees courses they are enrolled in.

Capability	Admin	Teacher	Student
Create/deactivate users, reset any password	Yes	No	No
Create courses, assign teachers, enroll students	Yes	No	No
Upload resources (own courses)	Yes	Yes	No
View/download resources (enrolled courses)	Yes	Yes	Yes
Create assignments and quizzes (own courses)	Yes	Yes	No
Submit assignments, attempt quizzes	No	No	Yes
Grade submissions	Yes	Yes (own courses)	No
View marks	All	Own courses	Own marks only
Post announcements	Yes (global)	Yes (own courses)	No
Read announcements	Yes	Yes	Yes (enrolled + global)
Classroom chat	Read all	Own courses	Enrolled courses
File a complaint	No	Yes	Yes
Resolve complaints	Yes	No	No

RLS approach:

- Enable RLS on every table. The API uses the user's JWT for reads where possible so RLS is the second line of defense; the service-role client is used only for admin actions (create user, reset password) and is never reachable from a non-admin route.
- Create two SQL helper functions: `is_enrolled(course_id)` and `teaches(course_id)` (both `security definer`, based on `auth.uid()`). Policies are written against these helpers.
- Quiz answer keys and other students' submissions must never be readable by students, even via a direct Supabase client call. Put correct answers in a separate table (`quiz_answers`) with a teacher/admin-only policy.

4. Database schema (Supabase Postgres)

The AI should turn this into ordered migration files (`0001_core.sql` , `0002_content.sql` , and so on), add RLS policies from Section 3, and add a `supabase/seed.sql` with 1 admin, 2 teachers, 5 students, 3 courses.

```
create extension if not exists pgcrypto;

create type user_role as enum ('admin','teacher','student');

-- Users: id matches auth.users.id
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null,
  email text not null unique,
  role user_role not null,
  roll_no text unique,                -- students only
  is_active boolean not null default true,
  must_change_password boolean not null default true,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table courses (
  id uuid primary key default gen_random_uuid(),
  code text not null unique,          -- e.g. CS101
  title text not null,
  description text,
  is_archived boolean not null default false,
  created_at timestamptz not null default now()
);

create table course_teachers (
  course_id uuid references courses(id) on delete cascade,
  teacher_id uuid references profiles(id) on delete cascade,
  primary key (course_id, teacher_id)
);

create table enrollments (
  course_id uuid references courses(id) on delete cascade,
  student_id uuid references profiles(id) on delete cascade,
  enrolled_at timestamptz not null default now(),
  primary key (course_id, student_id)
);

create table resources (
```

```
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  uploaded_by uuid not null references profiles(id),
  title text not null,
  description text,
  file_path text not null,            -- path in 'resources' bucket
  file_name text not null,
  mime_type text not null,
  size_bytes bigint not null,
  created_at timestamptz not null default now()
);

create table assignments (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  created_by uuid not null references profiles(id),
  title text not null,
  instructions text not null,
  max_marks numeric(6,2) not null check (max_marks > 0),
  due_at timestamptz not null,
  allow_late boolean not null default false,
  attachment_path text,
  created_at timestamptz not null default now()
);

create table submissions (
  id uuid primary key default gen_random_uuid(),
  assignment_id uuid not null references assignments(id) on delete cascade,
  student_id uuid not null references profiles(id) on delete cascade,
  text_answer text,
  file_path text,
  submitted_at timestamptz not null default now(),
  is_late boolean not null default false,
  marks_obtained numeric(6,2),
  feedback text,
  graded_by uuid references profiles(id),
  graded_at timestamptz,
  unique (assignment_id, student_id)
);

create table quizzes (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  created_by uuid not null references profiles(id),
  title text not null,
  description text,
```

```
  time_limit_minutes int not null check (time_limit_minutes > 0),
  opens_at timestamptz not null,
  closes_at timestamptz not null,
  is_published boolean not null default false,
  created_at timestamptz not null default now(),
  check (closes_at > opens_at)
);

create type question_type as enum ('mcq_single','mcq_multi','true_false');

create table quiz_questions (
  id uuid primary key default gen_random_uuid(),
  quiz_id uuid not null references quizzes(id) on delete cascade,
  position int not null,
  type question_type not null,
  prompt text not null,
  options jsonb not null,             -- [{"id":"a","text":"..."}]
  marks numeric(5,2) not null default 1
);

-- Teacher/admin only (RLS): correct option ids
create table quiz_answers (
  question_id uuid primary key references quiz_questions(id) on delete
cascade,
  correct_option_ids text[] not null
);

create table quiz_attempts (
  id uuid primary key default gen_random_uuid(),
  quiz_id uuid not null references quizzes(id) on delete cascade,
  student_id uuid not null references profiles(id) on delete cascade,
  started_at timestamptz not null default now(),
  submitted_at timestamptz,
  answers jsonb not null default '{}',  -- {question_id: [option_ids]}
  score numeric(6,2),
  unique (quiz_id, student_id)          -- one attempt per student
);

create table announcements (
  id uuid primary key default gen_random_uuid(),
  course_id uuid references courses(id) on delete cascade, -- null = global
  author_id uuid not null references profiles(id),
  title text not null,
  body text not null,
  is_pinned boolean not null default false,
  created_at timestamptz not null default now()
```

```
);

create table announcement_reads (
  user_id uuid references profiles(id) on delete cascade,
  announcement_id uuid references announcements(id) on delete cascade,
  read_at timestamptz not null default now(),
  primary key (user_id, announcement_id)
);

create table audit_log (
  id bigint generated always as identity primary key,
  actor_id uuid references profiles(id),
  action text not null,
  entity text not null,
  entity_id uuid,
  at timestamptz not null default now()
);

create table chat_messages (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  sender_id uuid not null references profiles(id),
  body text not null check (char_length(body) between 1 and 2000),
  created_at timestamptz not null default now(),
  deleted_at timestamptz
);

create type complaint_status as enum
('open','in_review','resolved','rejected');

create table complaints (
  id uuid primary key default gen_random_uuid(),
  raised_by uuid not null references profiles(id),
  course_id uuid references courses(id),
  category text not null,             -- academic, technical, facility,
conduct, other
  subject text not null,
  description text not null,
  attachment_path text,
  status complaint_status not null default 'open',
  admin_response text,
  resolved_by uuid references profiles(id),
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);
```

```
create index on enrollments (student_id);
create index on resources (course_id, created_at desc);
create index on assignments (course_id, due_at);
create index on submissions (assignment_id);
create index on quiz_attempts (quiz_id);
create index on announcements (course_id, created_at desc);
create index on chat_messages (course_id, created_at desc);
create index on complaints (status, created_at desc);
```

Marks are derived, not duplicated: assignment marks live in `submissions.marks_obtained`, quiz marks in `quiz_attempts.score`. Expose them through one view `v_student_marks` that unions both into `(student_id, course_id, kind, item_id, title, obtained, max, graded_at)`.

5. Feature specs 1–5

Each feature lists user stories, API, UI screens, rules, and acceptance criteria. All endpoints are prefixed `/api` and return JSON.

Feature 1: Login (no sign-up)

Stories: As any user I log in with email and password and land on my role's dashboard. As an admin I create user accounts (there is no public sign-up).

API:

Method	Path	Access	Purpose
POST	`/auth/login`	Public	Email + password, returns tokens and profile
POST	`/auth/logout`	Auth	Invalidate session
POST	`/auth/refresh`	Public	Exchange refresh token
GET	`/auth/me`	Auth	Current profile and role
POST	`/admin/users`	Admin	Create user (name, email, role, temp password), sets `must_change_password`
PATCH	`/admin/users/:id`	Admin	Edit name, role, activate/deactivate
GET	`/admin/users`	Admin	List, search, filter by role

UI: `/login` (email, password, show/hide toggle, error banner), role-based redirect (`/admin`, `/teacher`, `/student`), protected-route wrapper, admin Users page (table, create dialog, deactivate switch). Do NOT build a sign-up page or link.

Rules: rate-limit login to 5 attempts per 15 min per IP+email; generic error "Invalid email or password" (never reveal which part was wrong); deactivated users are blocked; first login with a temp password forces a password change.

Acceptance:

- [ ] Valid login redirects by role; invalid shows the generic error
- [ ] `/signup` returns 404 and Supabase sign-ups are disabled
- [ ] Deactivated user cannot log in or reuse an old token
- [ ] Visiting a protected route logged-out redirects to `/login`

Feature 2: Resources upload (teacher) and view (student)

The phrase "marks seen by students" is specified in Feature 7; this feature covers resources only.

Stories: As a teacher I upload files (PDF, DOCX, PPTX, images, ZIP) to my course with a title. As a student I browse and download resources of enrolled courses.

API:

Method	Path	Access	Purpose
POST	`/courses/:id/resources`	Teacher (own course)	Multipart upload plus title/description
GET	`/courses/:id/resources`	Enrolled / teaches	Paginated list, search by title
GET	`/resources/:id/download`	Enrolled / teaches	Returns a 60 s signed URL
DELETE	`/resources/:id`	Uploader / Admin	Deletes row and storage object

UI: course page "Resources" tab: teacher sees Upload button (drag-and-drop, progress bar) and delete icons; student sees a list with file icon, size, date, Download.

Rules: max 25 MB per file; allow-list of MIME types checked server-side (not only by extension); files are stored at `resources/{course_id}/{uuid}-{filename}`; bucket is private.

Acceptance:

- [ ] Teacher cannot upload to a course they do not teach (403)
- [ ] Student not enrolled gets 403 on list and download

- [ ] A 30 MB file or `.exe` is rejected with a clear message
- [ ] Deleting a resource removes the storage object

Feature 3: Course assignment to students

Stories: As an admin I create courses, assign teachers, and enroll students (single or bulk by CSV of emails). Students see "My courses"; teachers see "My classes".

API:

Method	Path	Access	Purpose
POST/GET/PATCH	`/admin/courses`	Admin	CRUD, archive instead of delete
PUT	`/admin/courses/:id/teachers`	Admin	Replace teacher set
POST	`/admin/courses/:id/enrollments`	Admin	Body: `{ studentIds[] }` or CSV upload
DELETE	`/admin/courses/:id/enrollments/:studentId`	Admin	Unenroll
GET	`/me/courses`	Student/Teacher	Courses for the current user

UI: admin Courses page, course detail with Teachers and Students tabs (multi-select picker, CSV import with a preview of unmatched emails), student dashboard course cards.

Acceptance:

- [ ] Enrolling the same student twice is idempotent (no error, no duplicate)
- [ ] CSV import reports matched, duplicate, and unknown emails
- [ ] Student sees only enrolled, non-archived courses

Feature 4: Assignments

Stories: As a teacher I create an assignment with instructions, optional attachment, due date, max marks. As a student I submit text and/or one file before the due date and can resubmit until the deadline.

API:

Method	Path	Access	Purpose
POST	`/courses/:id/assignments`	Teacher	Create
GET	`/courses/:id/assignments`	Enrolled / teaches	List with my submission status
PATCH/DELETE	`/assignments/:id`	Teacher (own)	Edit / delete
PUT	`/assignments/:id/submission`	Student	Create or replace own submission
GET	`/assignments/:id/submissions`	Teacher	All submissions with student names
PATCH	`/submissions/:id/grade`	Teacher	`{ marks, feedback }`

Rules: server decides lateness using `now() > due_at` (never trust the client clock); if `allow_late = false` after the deadline, reject with 409; `marks` must be between 0 and `max_marks`; resubmission clears prior grade.

UI: student list with status chips (Pending, Submitted, Late, Graded) and countdown; teacher grading table with inline marks input, feedback field, and a download link for each file.

Acceptance:

- [ ] Late submission blocked or flagged per `allow_late`
- [ ] Marks above `max_marks` rejected (422)
- [ ] Student can never read another student's submission

Feature 5: Quizzes

Stories: As a teacher I build a timed quiz of MCQ (single or multi) and true/false questions, publish it for a window. As a student I attempt it once within the time limit and see my score after submitting.

API:

Method	Path	Access	Purpose
POST	`/courses/:id/quizzes`	Teacher	Create quiz with questions and answers

Method	Path	Access	Purpose
PATCH	`/quizzes/:id`	Teacher	Edit (blocked once any attempt exists), publish/unpublish
GET	`/courses/:id/quizzes`	Enrolled / teaches	List with status
POST	`/quizzes/:id/attempt`	Student	Start: returns questions WITHOUT answers, `started_at`, `expires_at`
POST	`/quizzes/:id/attempt/submit`	Student	Submit answers; server grades
GET	`/quizzes/:id/results`	Teacher	All attempts and scores

Rules: server-side auto-grading against `quiz_answers`; multi-select is all-or-nothing per question; reject submissions after `started_at + time_limit` plus 10 s grace (auto-submit client-side at zero); only one attempt per student; shuffle question order per attempt (optional flag); never send correct answers to students.

UI: teacher quiz builder (add/reorder questions, mark correct options, preview); student quiz runner with a timer, question navigator, and confirm-submit dialog; a result screen showing the score.

Acceptance:

- [ ] Network inspection of the attempt response shows no answer key
- [ ] A second attempt is rejected (409)
- [ ] Submission after expiry is rejected or scored 0 for unanswered
- [ ] Score equals the sum of marks of fully-correct questions

6. Feature specs 6–10

Feature 6: Announcements

Stories: As a teacher I post announcements to one of my courses; as an admin I post global ones. Students see them in a feed, with unread badges.

Method	Path	Access	Purpose
POST	`/announcements`	Teacher/Admin	`{ courseId?, title, body, isPinned }`; null `courseId` is admin-only

Method	Path	Access	Purpose
GET	`/announcements`	Auth	Feed: global plus enrolled/taught courses, pinned first, paginated
PATCH/DELETE	`/announcements/:id`	Author/Admin	Edit / delete
POST	`/announcements/read`	Auth	Mark read up to a timestamp (stored in a `announcement_reads` table: user_id, announcement_id)

UI: dashboard widget (latest 5) plus a full feed page, course filter, unread badge in the navbar updated through Realtime.

Acceptance:

- [ ] A student sees global plus own-course announcements only
- [ ] A teacher cannot post to a course they do not teach or post global
- [ ] Pinned items always sort first

Feature 7: Marks for assignments and quizzes

Stories: As a student I see all my marks per course (assignments and quizzes) with a total and percentage. As a teacher I see a gradebook for my course. As an admin I can see everything.

Method	Path	Access	Purpose
GET	`/me/marks?courseId=`	Student	Own marks from `v_student_marks`
GET	`/courses/:id/gradebook`	Teacher/Admin	Matrix: students by items, with totals
GET	`/courses/:id/gradebook.csv`	Teacher/Admin	CSV export

Rules: percentage is `sum(obtained) / sum(max) * 100`, counting only graded items; ungraded items show "Pending" and are excluded from totals; marks are visible to the student only after grading is saved.

UI: student "My Marks" page (table per course, progress bar, totals row); teacher gradebook (sticky first column, click a cell to jump to grading).

Acceptance:

- [ ] A student can never request another student's marks (403)
- [ ] Totals match a manual calculation on seeded data
- [ ] CSV opens correctly in Excel (UTF-8 BOM)

Feature 8: Classroom chat

Stories: Every course has one group chat for its teachers and enrolled students. Messages appear instantly.

Method	Path	Access	Purpose
GET	`/courses/:id/messages?before=&limit=50`	Member	Cursor-paginated history
POST	`/courses/:id/messages`	Member	`{ body }`, 1–2000 chars
DELETE	`/messages/:id`	Sender (5 min) / Teacher / Admin	Soft delete (`deleted_at`)

Realtime: the React client subscribes to Supabase Realtime `postgres_changes` (INSERT and UPDATE) on `chat_messages` filtered by `course_id`; RLS must restrict the subscription to members. Sending goes through the API so validation and rate limits (20 messages/min/user) apply.

UI: left list of the user's courses, right message pane with sender name and role badge, infinite scroll upward, auto-scroll to bottom when already at the bottom, "deleted message" placeholder, optimistic send with retry.

Acceptance:

- [ ] A message sent by user A appears for user B within 2 s without refresh
- [ ] A non-member cannot read, send or subscribe
- [ ] HTML in the body is rendered as text (no XSS)

Feature 9: Register a complaint

Stories: As a student or teacher I raise a complaint with a category, subject and description (optional attachment, optional course) and track its status. As an admin I review, respond, and change the status.

Method	Path	Access	Purpose
POST	`/complaints`	Student/Teacher	Create (multipart for attachment, max 10 MB)
GET	`/complaints/mine`	Student/Teacher	Own complaints and statuses
GET	`/admin/complaints?status=&category=`	Admin	Queue with filters
PATCH	`/admin/complaints/:id`	Admin	`{ status, adminResponse }`

Rules: status flow `open → in_review → resolved | rejected`; a response is required when resolving or rejecting; the complainant gets an in-app notice (new announcement-style badge) on a status change; users cannot edit a complaint after filing but can add one follow-up comment (optional extension).

UI: a "Raise a complaint" form, a "My complaints" list with status chips, an admin table with a detail drawer and response box.

Acceptance:

- [ ] A user sees only their own complaints; other users get 403/404
- [ ] Resolving without a response is rejected (422)
- [ ] Attachment is stored in the private bucket and delivered by a signed URL

Feature 10: Reset password

Stories: As a user who forgot their password I request a reset link by email. As a logged-in user I change my password. As an admin I can trigger a reset for any user.

Method	Path	Access	Purpose
POST	`/auth/forgot-password`	Public	`{ email }`, always returns 200 with the same message
POST	`/auth/reset-password`	Recovery token	`{ newPassword }` (token from the email link)
POST	`/auth/change-password`	Auth	`{ currentPassword, newPassword }`
POST	`/admin/users/:id/reset-password`	Admin	Sends a reset email (or sets a temp password)

Rules: never reveal whether an email exists; reset links expire in 1 hour and are single-use; password policy: at least 10 characters, one letter and one number; rate-limit `forgot-password` to 3 per hour per email; after a successful reset, revoke all other sessions.

UI: `/forgot-password` (email field and neutral confirmation), `/reset-password` (new plus confirm, strength meter, expired-link state), a Change Password section on the profile page.

Acceptance:

- [ ] Unknown email and known email show identical responses and similar timing
- [ ] An expired or reused link shows a clear error with a link to request a new one
- [ ] The old password stops working immediately after a reset

7. API conventions, structure, and non-functional requirements

Conventions:

- Success: `{ "data": ..., "meta": { "page", "pageSize", "total" } }`. Error: `{ "error": { "code": "FORBIDDEN", "message": "...", "details": [...] } }`.
- Status codes: 200/201 success, 400 malformed, 401 not logged in, 403 wrong role or not a member, 404 not found (also used to hide existence), 409 conflict (duplicate, late, second attempt), 422 validation failure, 429 rate limited.
- Pagination: `?page=1&pageSize=20` (max 100); chat uses cursor `?before=<iso>`.
- JSON keys are camelCase in the API and snake_case in the DB; map in one place (the repository layer).
- Layering in the API: `routes → controllers → services → repositories`. No Supabase calls inside controllers.

Folder structure (monorepo):

```
lms/
  apps/
    web/                      # React + Vite
      src/
        app/                  # router, providers, layouts
        features/
          auth/ courses/ resources/ assignments/ quizzes/
          announcements/ marks/ chat/ complaints/ admin/
        components/ui/        # shadcn components
        lib/                  # api client, supabase client, utils
    api/                      # Node + Express
      src/
        config/ middleware/ routes/ controllers/
        services/ repositories/ schemas/ (zod) utils/
      tests/
  supabase/
    migrations/ seed.sql config.toml
  docs/ASSUMPTIONS.md
  README.md
```

Security:

- Helmet, strict CORS to `CLIENT_ORIGIN`, body size limit 1 MB (uploads via multer with per-route limits).
- Passwords are handled only by Supabase Auth; never log tokens, passwords, or full request bodies.
- Sanitize and escape all user text on render (React default); no `dangerouslySetInnerHTML`.
- Signed URLs for every file; all buckets private; validate MIME by magic bytes.
- Service-role key lives only in the API environment.

Quality and performance:

- Responsive from 360 px wide; WCAG AA contrast; full keyboard navigation; labels on all inputs.
- List pages respond in under 500 ms for 5,000 rows (indexes in Section 4); no N+1 queries.
- Audit trail: log who graded, who deleted what, and admin user changes to an `audit_log` table (actor_id, action, entity, entity_id, at).
- Provide `npm run dev`, `npm run test`, `npm run db:reset` (migrations plus seed), and a README with setup in 10 steps or fewer.

8. Build order and AI prompts

Build in six phases. Paste this doc (or the relevant sections) into the AI together with the phase prompt. After each phase, tick the acceptance boxes before continuing.

Phase	Scope	Depends on
0	Monorepo scaffold, Supabase project, migrations, seed, CI	none
1	Feature 1 (Login), Feature 10 (Reset password), admin user management	0
2	Feature 3 (Courses and enrollment), Feature 2 (Resources)	1
3	Feature 4 (Assignments), Feature 5 (Quizzes), Feature 7 (Marks)	2
4	Feature 6 (Announcements), Feature 8 (Chat)	2
5	Feature 9 (Complaints), polish, audit log, E2E tests, README	3, 4

Phase 0 prompt:

```
You are a senior full-stack engineer. Read the attached "Internal LMS – AI
Build Spec".
Task: Phase 0 only. Create the monorepo (apps/web Vite+React+TS, apps/api
Express+TS),
lint/prettier/tsconfig, the Supabase migrations from Section 4 split into
ordered files,
RLS policies from Section 3, seed.sql, .env.example files, and npm scripts
listed in Section 7.
Do not build any features yet. Finish by listing what you created and any
assumptions
you wrote to docs/ASSUMPTIONS.md.
```

Template for phases 1–5:

```
Read the attached spec. Implement Phase <N>: Features <list> exactly as
specified in
Section 5/6, following the rules in Section 1 and conventions in Section 7.
For each feature deliver: migration changes (if any), RLS, API routes with
Zod schemas,
service/repository code, React screens with loading/empty/error states, and
tests.
After coding, run the tests and report each acceptance criterion as PASS or
FAIL with evidence.
Fix every FAIL before stopping. Do not implement anything outside this phase.
```

Review prompt (run after every phase):

```
Act as a security and QA reviewer. For the code added in this phase, try to
break authorization:
list every endpoint, and for each one state what a student, a teacher of
another course,
and an unauthenticated user get. Check for missing RLS, missing Zod
validation,
leaked answer keys, and unbounded queries. Report findings by severity and
fix the high ones.
```

Final acceptance checklist

- [ ] All 10 features meet their acceptance criteria
- [ ] No sign-up route or UI exists anywhere
- [ ] A student cannot read another student's marks, submissions, or complaints, nor any quiz answer key (verified by tests)
- [ ] RLS is enabled on every table; the service-role key is absent from the web bundle
- [ ] Seeded demo data lets a reviewer walk all three roles end to end
- [ ] `npm run test` is green; one Playwright flow per role passes
- [ ] README setup works on a clean machine

Open items to confirm

- [ ] Is quiz retake needed? (Spec assumes one attempt per student.)
- [ ] Should chat support files or images? (Spec: text only.)
- [ ] Should marks release automatically on grading, or after a teacher publishes them? (Spec: on save.)
- [ ] Notifications by email, or in-app only? (Spec: in-app only.)